

# Allgemeine Hinweise

---

## Web-Engineering II

Prof. Dr. Sebastian von Klinski

---

---

# Visual Studio Code

- Freier Quelltext-Editor von Microsoft
- Plattformübergreifend für die Betriebssysteme Windows, macOS und Linux
- Basiert auf dem Framework Electron
- Hat nichts mit Visual Studio zu tun!!!
- Wird hauptsächlich von einem Team in der Schweiz entwickelt, das von Erich Gamma geleitet wird!!!
- Kann durch zahllose Plug-ins erweitert werden (Extensions)
- Unter anderem für
  - Note.js
  - React/ React Native
  - Angular
  - Vue.js
  - Etc.

---

# Node.js

- Muss separat zu Visual Studio Code installiert werden
- Nach Installation von Node.js kann Package-Manager npm verwendet werden
- npm- Befehle
  - Datei package.json anlegen
    - ❖ npm init
  - React-Projekt anlegen
    - ❖ npm init react-app ./my-react-app
  - Installation von allen Pakten, die im package.json angegeben sind.
    - ❖ npm install

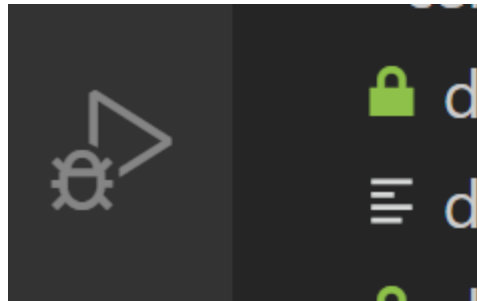
---

# **Debugger für Node.js in VSC aufsetzen**

---

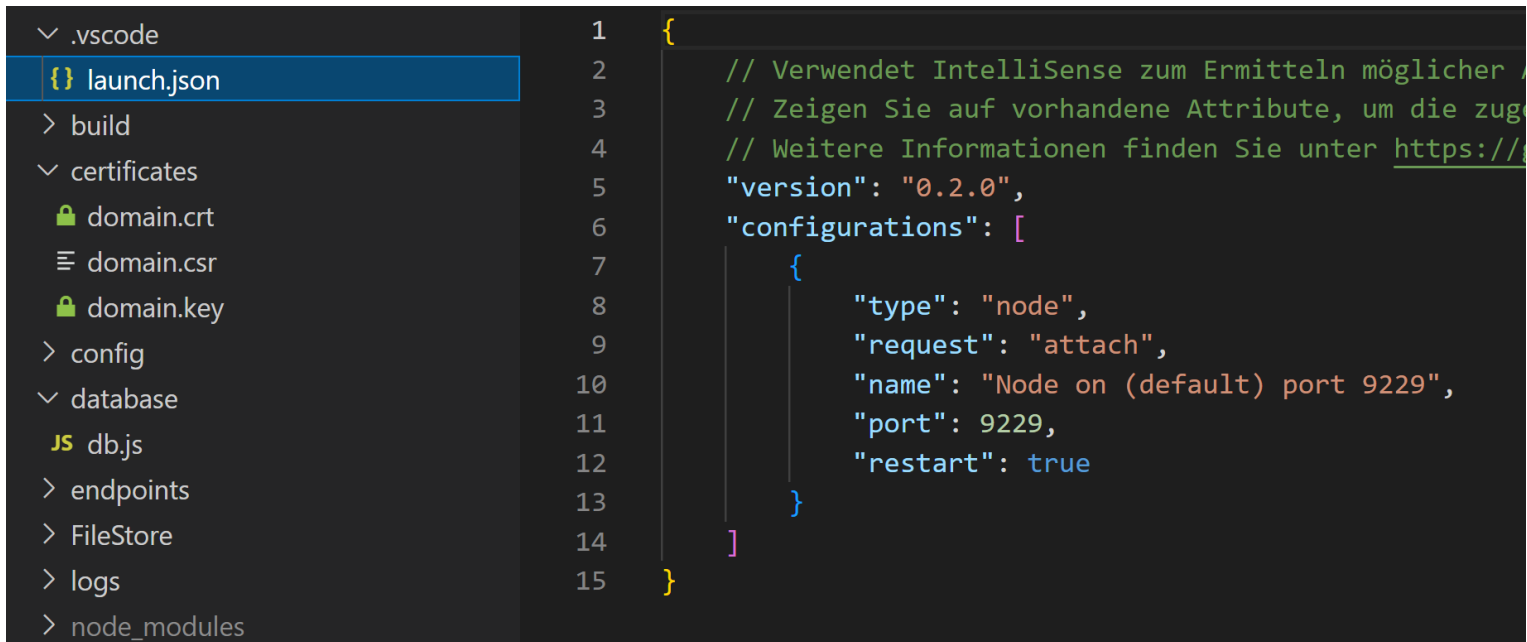
## Debugger in VSC

- Auch JavaScript-Programme können unter Visual Studio Code per Debugger analysiert werden
- Öffnen des Debugger-Menüs über den Activity-Bar
- Nutzung setzt ein paar Konfigurationsschritte voraus



# Schritt 1: Debug-Launch-Konfiguration anlegen

- .vscode/launch.json erstellen
- Im Debug-Menü kann Date auch automatisch erstellt werden
- Dort legt man folgende Konfiguration an
- Konfiguration sollte den folgenden Code beinhalten



```
1  {
2      // Verwendet IntelliSense zum Ermitteln möglicher
3      // Zeigen Sie auf vorhandene Attribute, um die zug
4      // Weitere Informationen finden Sie unter https://
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "type": "node",
9              "request": "attach",
10             "name": "Node on (default) port 9229",
11             "port": 9229,
12             "restart": true
13         }
14     ]
15 }
```

---

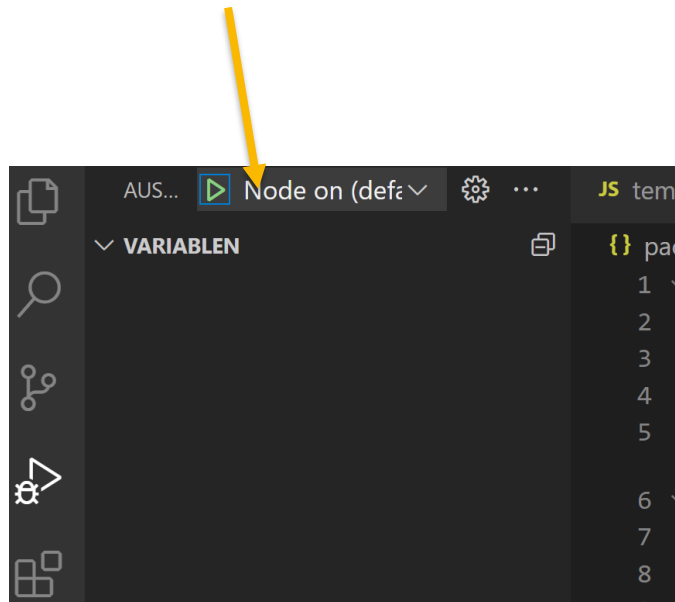
## Schritt 2: Test-Start-Skript anlegen

- Ein neues Skript in der package.json anlegen, um den Server im Debug-Mode zu starten:
- "startDev": "nodemon --inspect meinServer.js",

```
"scripts": {  
  "start": "nodemon ./httpServer.js",  
  "startDev": "nodemon --inspect ./httpServer.js",  
  "test": "echo \"Error: no test specified\" && exit 1"  
}
```

## Schritt 3: Server und Debugger starten

- Es müssen sowohl der REST-Server als auch der Debugger gestartet werden!
- Der Debugger überwacht den Server auf Event, läuft aber separat
  - Server starten: `npm run startDev`
  - Debugger starten: Button im Debugger-Menü drücken



---

## Schritt 4: Break-Points setzen und Anfragen abschicken

- Sie können jetzt im Programm-Code Break-Points setzen, indem Sie links von der Zeilennummer klicken → Es erscheint dann ein roter Punkt
- Mit den Debug-Tools oder den Funktionstasten können Sie dann schrittweise durch den Programmablauf gehen

```
13  
● 14  const testUser = new User({ userID: 'Hucky', userName: "Bodo Backer", password: "asdf" });  
15
```

## Weitere Hinweise

- Manchmal setzt VSC den Debugger falsch auf
- Wichtig ist, dass der Debugger den aktuellen Code und nicht die JavaScript-Dateien im “dist“-Verzeichnis verwendet.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug Express Server",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/src/server.ts",
      "runtimeArgs": ["-r", "ts-node/register"],
      "env": {
        "NODE_ENV": "development",
        "TS_NODE_PROJECT": "${workspaceFolder}/tsconfig.json"
      },
      "console": "integratedTerminal",
      "restart": true,
      "protocol": "inspector",
      "skipFiles": ["<node_internals>/**"],
      "resolveSourceMapLocations": [
        "${workspaceFolder}/**",
        "!**/node_modules/**"
      ],
      "sourceMaps": true,
      "outFiles": []
    }
  ],
}
```

---

# Logging

---

## Logging

- Zur Analyse vom Programmablauf kann neben Debugging auch Logging verwendet werden
- Häufig wird `console.log` für Logging verwendet
- `Console.log` hat mehrere Nachteile
  - Kein Ein- und Ausschalten von Logs
  - Keine Anpassung der Log-Ausgaben möglich
  - In Produktiv-Systemen beeinträchtigen Konsole-Ausgaben die Performance
- Besser ist Verwendung von Logging-Bibliothek/ -Framework

---

## Winston

- Im JavaScript-Bereich sehr verbreitete Logging-Bibliothek
- Installation: „npm install winston“
- Konfiguration über TypeScript-Datei
  - Definition von Log-Channels (Konsole, E-Mail, etc.)
  - Definition von Ausgabeformat
  - Anpassung von Log-Level an Umgebung (dev, prod, etc.)
- Logger muss aus Konfigurationsdatei importiert werden

```
import { IUser, IUserCreateRequest, IUserUpdateRequest } from './models';  
import { logger } from '../utils/logger';  
import User from './UserModel';
```

# Winston: Konfiguration

```
const consoleFormat = winston.format.combine(
  winston.format.errors({ stack: true }),
  winston.format.splat(),
  winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss.SSS' }),
  winston.format.colorize({ all: false, level: true }),
  winston.format.printf((info) => {
    const { timestamp, level, message, stack } = info;
    const text = typeof message === 'string' ? message : String(message);
    let line = `${String(timestamp)} ${String(level)}: ${text}`;
    if (typeof stack === 'string' && stack.length > 0) {
      line += `\n${stack}`;
    }
    return line;
  })
);
```

---

# Statische Code Analyse

---

## Statische Code Analyse

- Sehr hilfreich, um Fehler und schlechten Code automatisch zu finden
- Erhöht Qualität des Programm-Codes deutlich!
- Gängiges Tool im JavaScript-Umfeld: ESLint
- Verwendung
  - Installation von notwendigen Modulen (eslint, eslint/js, etc.)

```
npm install --save-dev eslint @eslint/js @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

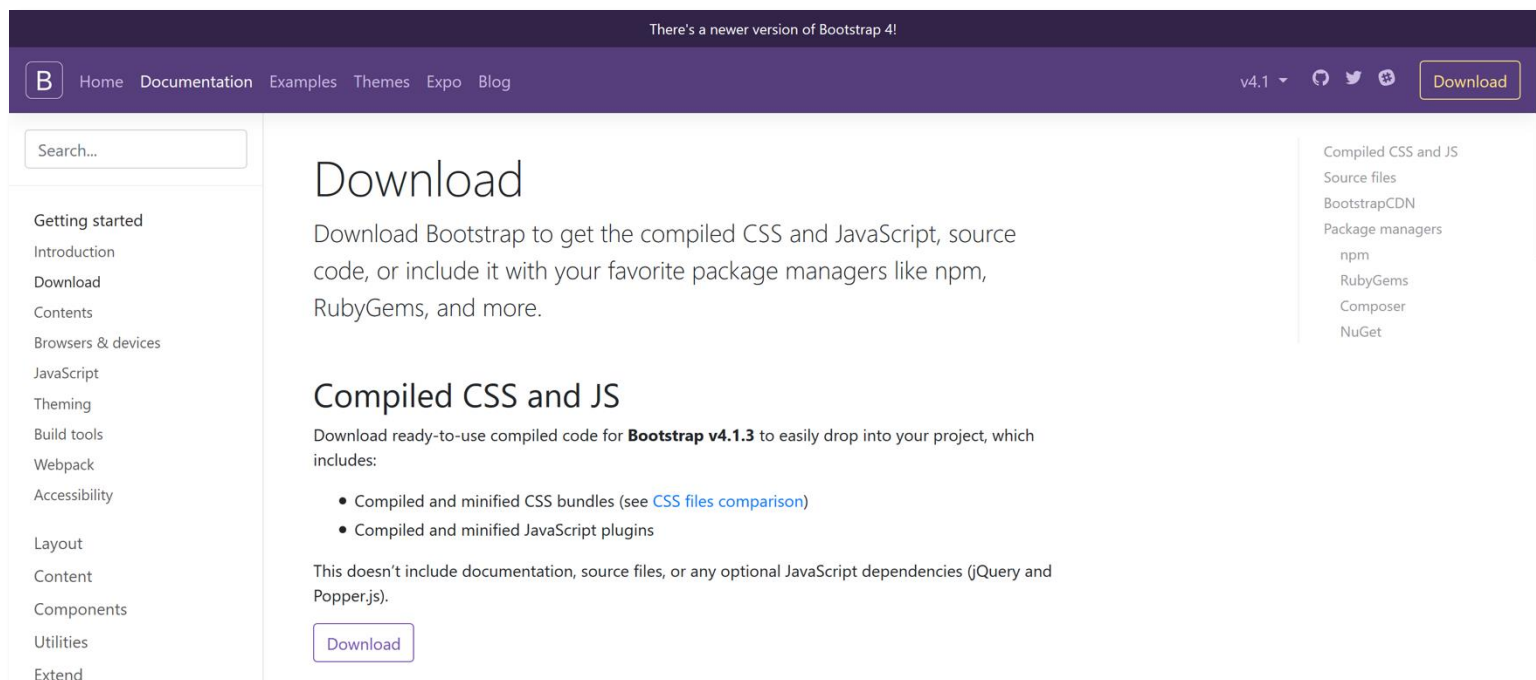
- Konfigurationsdatei `eslint.config.js` hinzufügen
- Bei Bedarf Skripte hinzufügen (lint: `eslint .`)
- Test, ob alles ok ist: `npx eslint .`

---

# Bootstrap

# Bootstrap

- Beinhaltet CSS- und JavaScript-Dateien
- Tutorials beziehen sich häufig auf unterschiedliche Versionen (nicht immer kompatibel)
- Download: <https://getbootstrap.com/docs/4.1/getting-started/download/>



---

# Bootstrap: Einbinden

## Empfehlung

- Legen Sie alle verwendeten CSS- und JavaScript-Dateien immer auf dem eigenen Server ab
- Einbinden von CSS- und anderen Dateien von anderen Servern datenschutzrechtliche problematisch
- Beim Abruf der Dateien können Anbieter Zugriffe loggen → Verstößt z.T. gegen DSGVO

```
...  
<head>  
...  
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">  
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>  
  <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/popper.min.js"></script>  
  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>  
</head>  
<body>
```

---

# Bootstrap: Einbinden

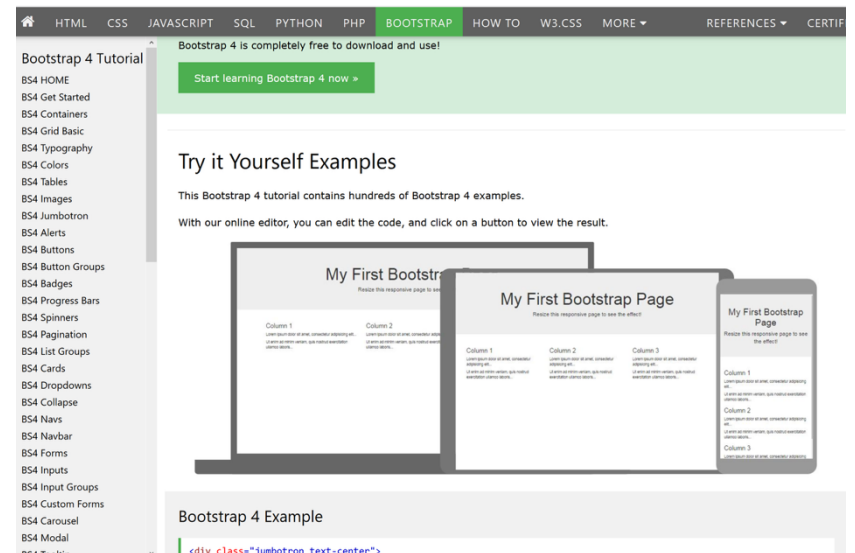
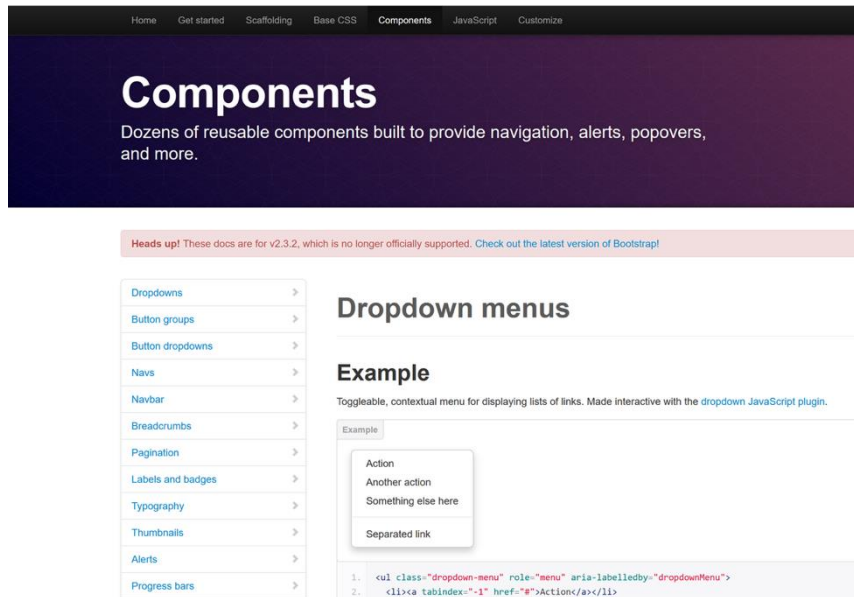
## Besser

- Alle CSS-, JavaScript- und Bilddateien lokal ablegen und einbinden

```
...  
<head>  
  <title>Bootstrap Example</title>  
  <meta charset="utf-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1">  
  <link rel="stylesheet" href="./lib/css/bootstrap.min.css">  
  <script src="./lib/js/jquery-3.4.1.min.js"></script>  
  <script src="./lib/js/popper.min.js"></script>  
  <script src="./lib/js/bootstrap.min.js"></script>  
</head>
```

# Bootstrap: Beispiele

- Im Internet viele gute Beispiele für Verwendung von Komponenten
- Komponentenübersichten
  - <https://getbootstrap.com/2.3.2/components.html>
  - <https://www.w3schools.com/bootstrap4/>



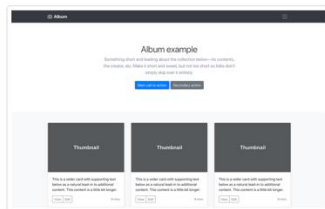
# Bootstrap: Beispiele

- Beispiele für ganze Seiten
  - <https://getbootstrap.com/docs/4.0/examples/>



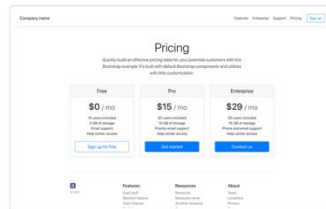
## Custom components

Brand new components and templates to help folks quickly get started with Bootstrap and demonstrate best practices for adding onto the framework.



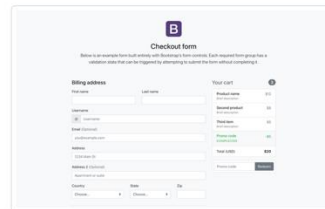
### Album

Simple one-page template for photo galleries, portfolios, and more.



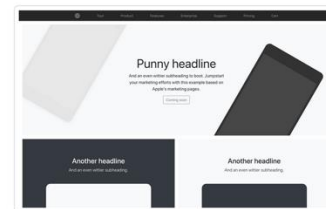
### Pricing

Example pricing page built with Cards and featuring a custom header and footer.



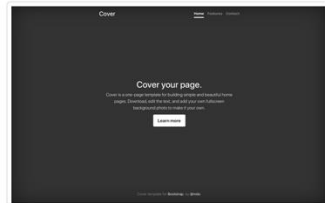
### Checkout

Custom checkout form showing our form components and their validation features.



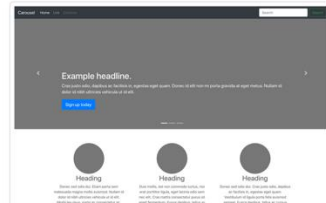
### Product

Lean product-focused marketing page with extensive grid and image work.



### Cover

A one-page template for building simple and beautiful home pages.



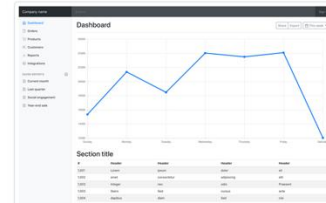
### Carousel

Customize the navbar and carousel, then add some new components.



### Blog

Magazine like blog template with header, navigation, featured content.



### Dashboard

Basic admin dashboard shell with fixed sidebar and navbar.

---

## Bootstrap: Einbinden

- Einbindung über npm nur sinnvoll bei npm-basierten Anwendungen (z.B. React, Express-Anwendungen, Webpack, etc.)
- Einbindung in Seiten über `require('bootstrap')`